Analysis of 8-bit arithmetic logic unit

Petro Nazeh Ozores Ghali

عبد الرحمن عوض عبد الحميد دندش

Ahmed Anas Elazab Elshehaby

Omar Dyaa

Supervisor : TA/ENG Hassan

Section : 5

Group : 9

ID : v23010552

20/12/2023

Contents

1. **Introduction**.....3
   1.1 8-BIT ALU Overview.....3
   1.2 Analysis of 8-bit ALU.....5
2. **Design**.....11
   2.2 ALU 8-bit behavioural module.....11
   2.3 ALU 8-bit structural module.....17
   2.4 RTL & Syntheses 26
3. **Testing & Simulation**.....29
   3.1 8-BIT ALU testbench.....30
   3.2 Simulation waveform output.....34
4. **Power & Timing analysis**.....35
5. **Refrences**.....39

1. Introduction

1.1 8-BIT ALU Overview

An 8-bit Arithmetic Logic Unit (ALU) is a digital circuit that performs arithmetic and logic operations on 8-bit binary numbers. The ALU typically includes functions such as addition, subtraction, AND, OR, XOR, shift left, shift right, and more. Each function is controlled by a set of control lines that determine the operation to be executed.

The ALU's output is determined by the specific operation selected and is usually represented as an 8-bit result. Control lines and flags help manage the operations and provide information about the results, such as overflow or zero flags.

8bit ALU Op-codes

S0	S1	S2	S3	F	Description
0	0	0	0	A+B	Add
0	0	0	1	A-B	Subtract
0	0	1	1	B'+1	2's complement
1	0	0	0	A and B	And
1	0	1	0	A xor B	Xor
1	0	0	1	A or B	Or
1	0	1	1	B'	1's complement
1	1	0	0	A>>	Right rotate
1	1	1	0	<<A	Left rotate
1	1	0	1	A>	Right shift
1	1	1	1	<B	Left shift

In summary, an 8-bit ALU is a versatile component in digital circuits, capable of performing a variety of arithmetic and logic operations on 8-bit binary data.

Key functions of an 8-bit ALU include:

1. **Addition and Subtraction:** The ALU can add two 8-bit numbers and perform subtraction by using additional control signals.

2. **Logical Operations:**

   - **AND Operation:** Bitwise AND operation between two 8-bit inputs.

   - **OR Operation:** Bitwise OR operation between two 8-bit inputs.

   - **XOR Operation:** Bitwise exclusive OR operation between two 8-bit inputs.

3. **Shift Operations:**

   - **Shift Left:** Shifts the bits of the input to the left.

   - **Shift Right:** Shifts the bits of the input to the right.

4. **Rotate Operations:**

   - **Rotate Left (ROL):** Circularly shifts the bits of the input to the left, with the leftmost bit wrapping around to the rightmost bit.

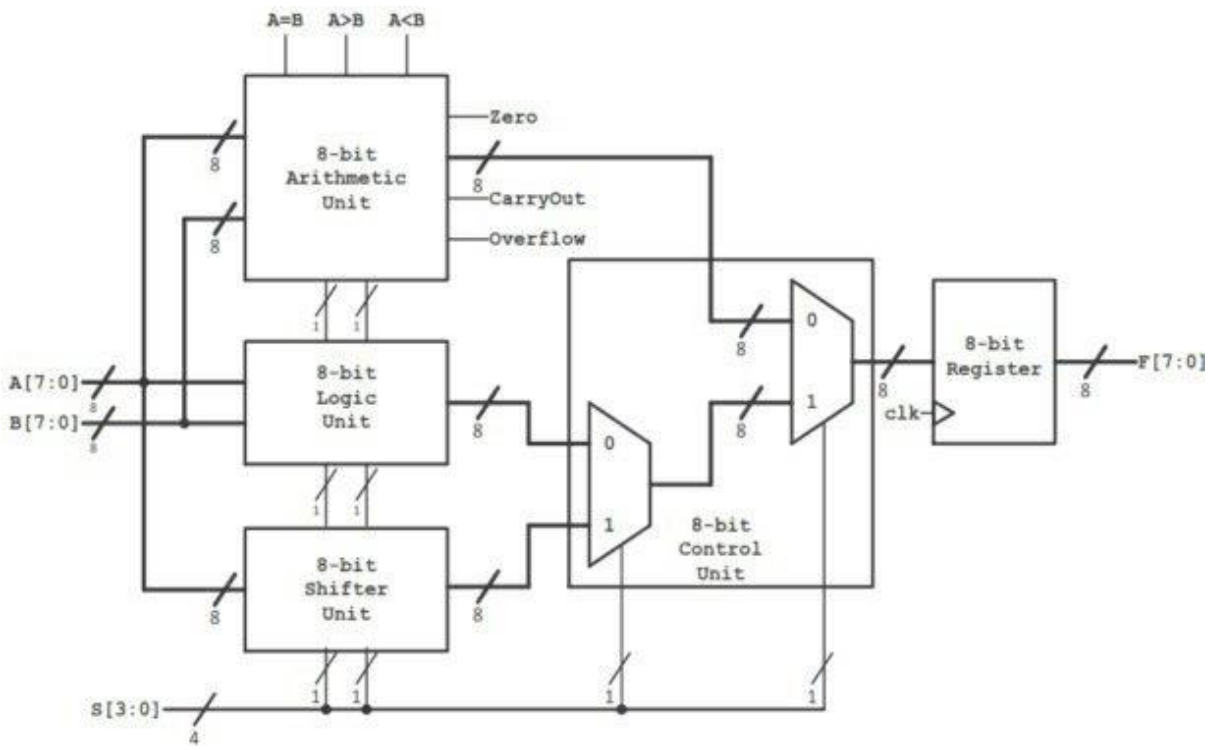
   - **Rotate Right (ROR):** Circularly shifts the bits of the input to the right, with the rightmost bit wrapping around to the leftmost bit.

5. **Comparison:** The ALU can generate flags indicating whether two numbers are equal, greater than, or less than each other.

6. **Bitwise NOT Operation:** Inverts each bit of the input (1’s complement) .

7. **2’s complement**

1.2 Analysis of 8-bit ALU



The 8bit ALU consists of five main blocks :-

1. **8-bit Arithmetic logic unit (8-bit signed comparator)**
2. **8-bit logic unit (8-bit Logic operations)**
3. **8-bit Shifter Unit**
4. **8-bit Register**
5. **Two 8-bit 2->1 mux**

1. 8-bit Arithmetic logic unit (8-bit signed comparator)

I/O

2 8-bit input ports (A,B) present data

1 2-bit input port (sel) present op selection

1 8-bit output port present result

6 1-bit output ports present flags

Functionality

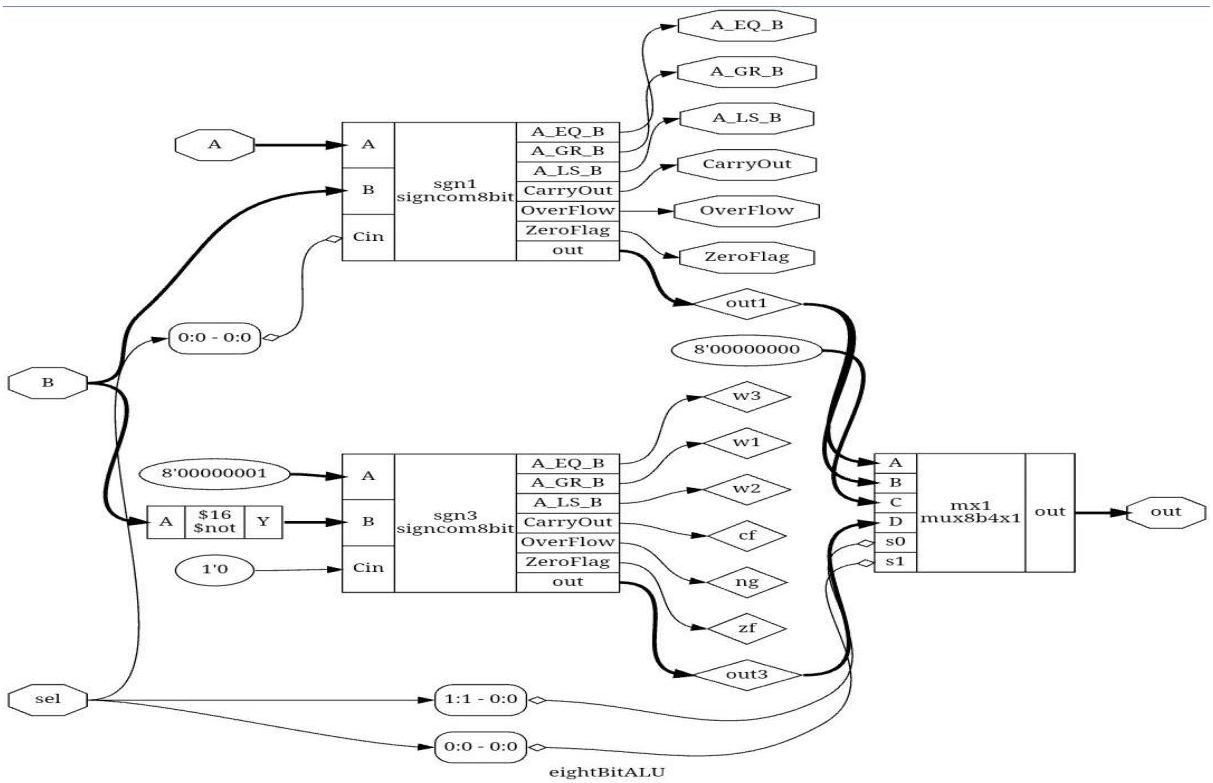
Depend on operation required it can be controlled using selector to achieve following functions Adding/Subtracting/2's complement for B

Implementation

It could be implemented using 8-bit signed comparator and one 8bit 4-1 mux

Using 1bit of sel we can choose add/sub operations and the other bit to

Out the 2's complement of B



2. 8-bit logic unit (8-bit Logic operations)

I/O

2 8-bit input ports (A,B) present data

1 2-bit input port (sel) present op selection

1 8-bit output port present result obtained

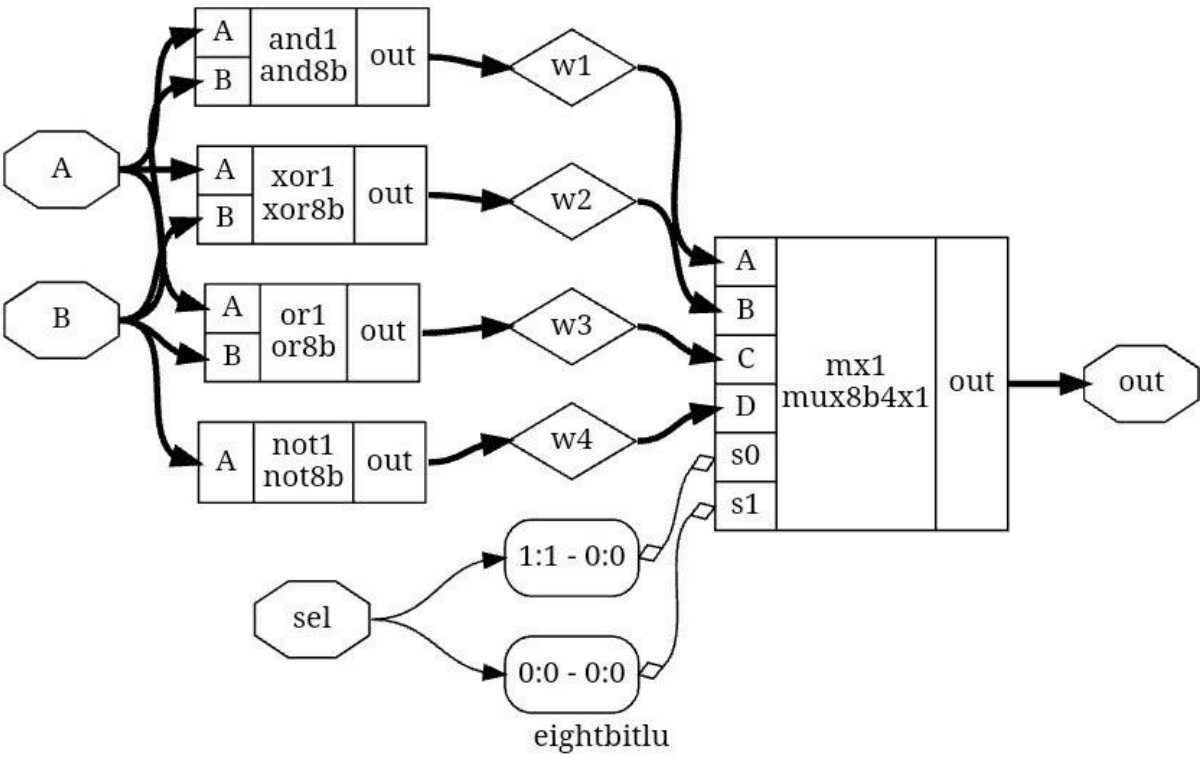
Functionality

Depend on operation required it can be controlled using selector to achieve following functions AND/OR/XOR/NOT(B')

Implementation

It could be implemented using 4 8-bit and/or/xor/not logic gates

and one 8bit 4-1 mux using 2 bit selector we can choose required operation



3. 8-bit Shifter unit

I/O

1 8-bit input ports (A) present data

1 2-bit input port (sel) present op selection

1 8-bit output port present result optained

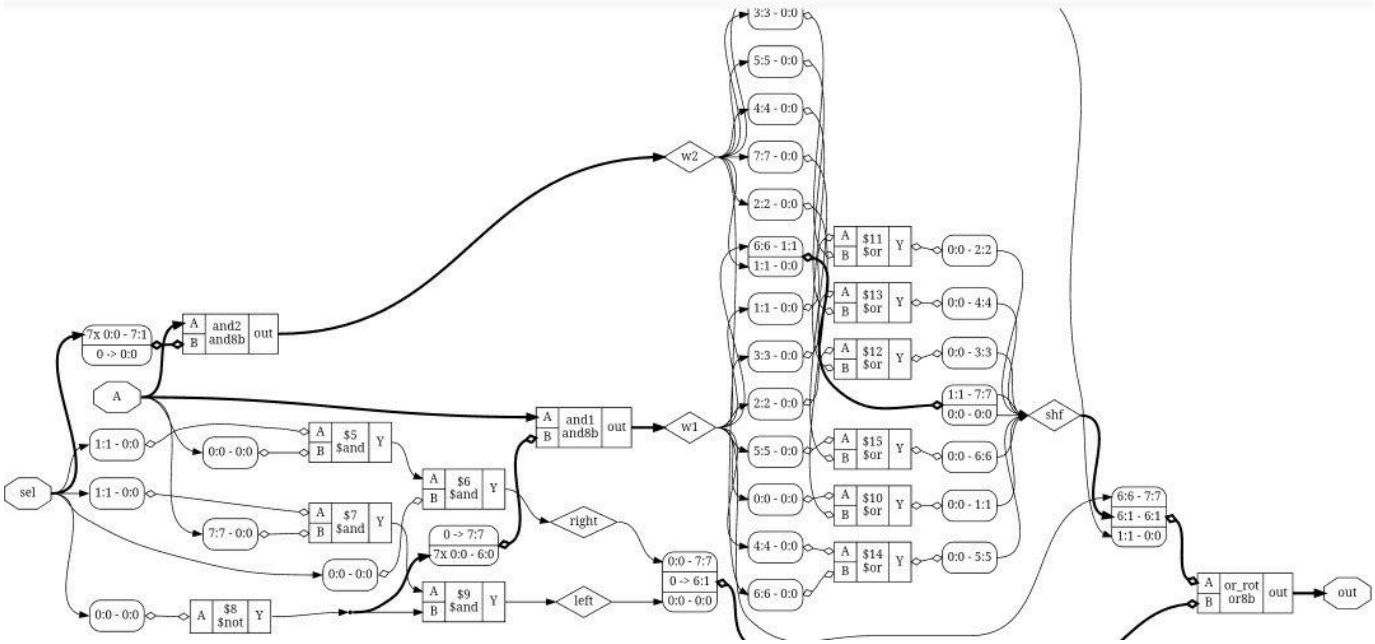
Functionality

Depend on operation required it can be controlled using selector to achieve following Rotate and shift right/left for A

Implementation

It could be implemented using and/or logic gates 1-bit sel for choose operation

Right or left and the other bit for selection direction left or right



4. 8-bit Register

I/O

1 8-bit input ports (A) present data

2 1-bit input for (negative) reset and read/write

1 8-bit output port for result

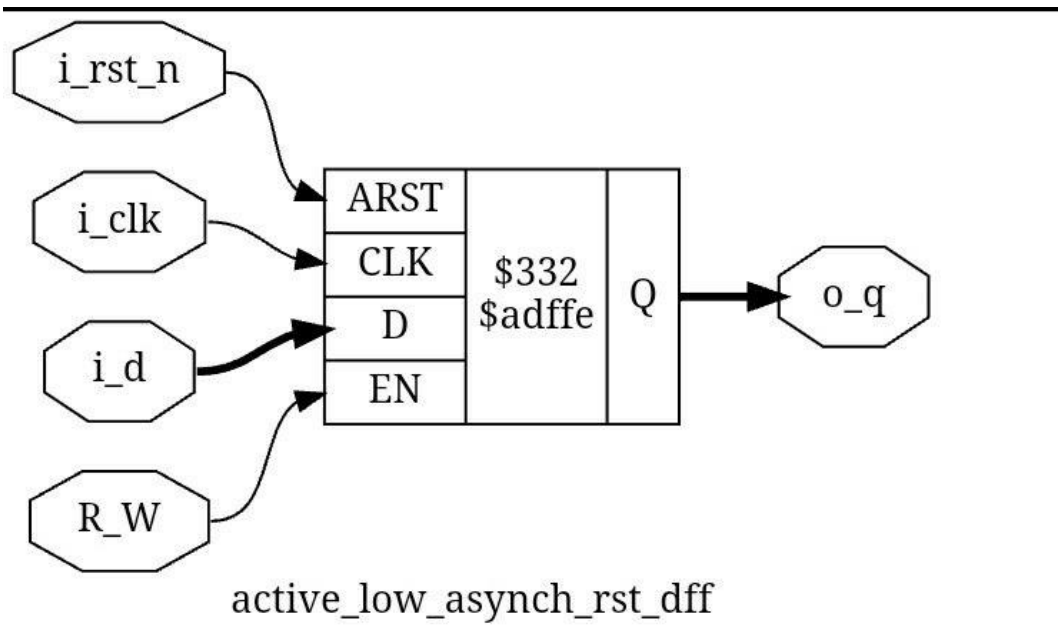
1 1-bit CLK input

Functionality

It saves the output from the ALU into 8bit memory depend on read/write op Required and CLK

Implementation

It could be implemented using eight 1-bit memory flip flop to make Register



5. Two 8-bit 2->1 mux

I/O

2 8-bit input ports (A,B) present data

1 2-bit input port (sel) present op selection

1 8-bit output port present result obtained

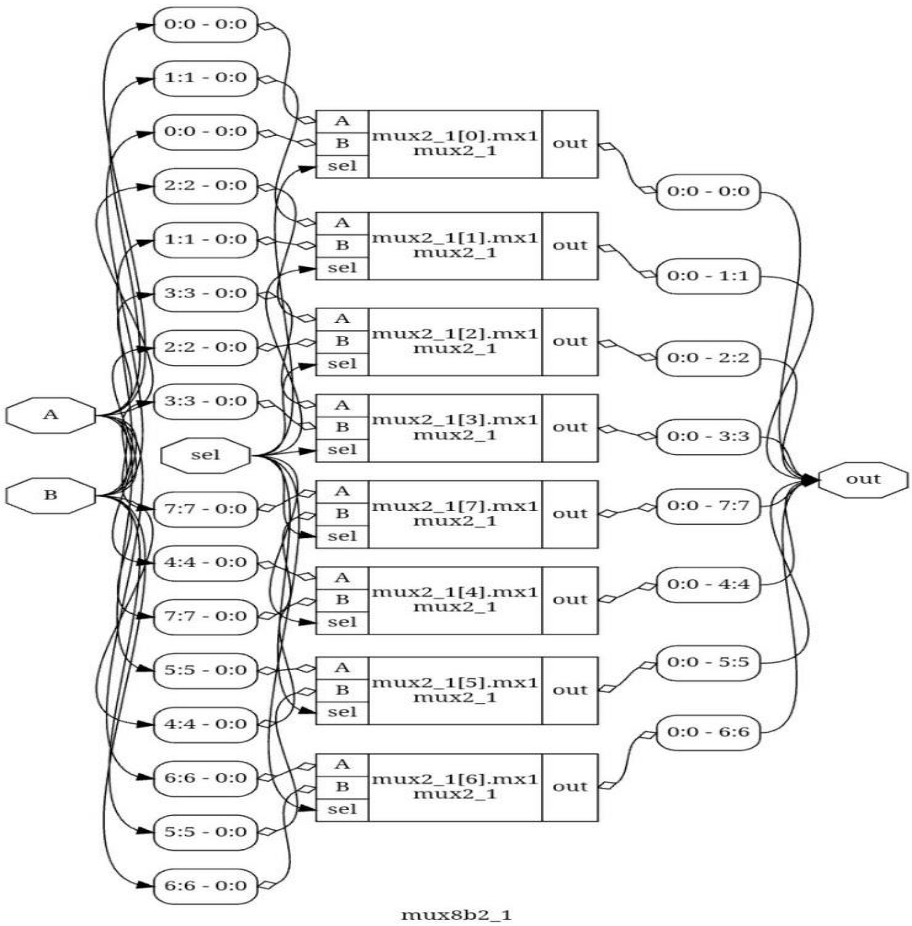
Functionality

Selects the output from the alu,logic unit/shifter unit depend on the sel input

Provided to be the final result obtained

Implementation

It could be implemented using eight 1 bit 2-1 mux



2. Design

2.1 ALU 8 bit behavioural module

```
module alu8bit (
input [7:0] X,
input [7:0] Y,
input [3:0] sel,
input i_clk,
input  R_W,
input i_rst_n,
output  [7:0] out,
output  OverFlow,
output  CarryOut,
output  ZeroFlag,
output  A_GR_B,
output  A_LS_B,
output  A_EQ_B
);

wire [7:0] w1;
wire [7:0] w2;
wire [7:0] w3;
wire [7:0] w4;
wire [7:0] w5;

eightBitALU alu(X,Y,{sel[1],sel[0]},OverFlow,CarryOut,ZeroFlag,A_GR_B,A_LS_B,A_EQ_B,w1);
eightbitlu el(X,Y,{sel[1],sel[0]},w2);
eightbitshfun sh(X,{sel[1],sel[0]},w3);

mux16_8 uy(w2,w3,~sel[2],w4);
```

```
mux16_8 ux(w4,w1,sel[3],w5);
active_low_asynch_rst_dff rg(out,R_W,w5,i_clk,i_rst_n);
endmodule
```

```
module eightBitALU (
   input wire [7:0] A,
   input wire [7:0] B,
   input wire [1:0] sel,
   output reg OverFlow,
   output reg CarryOut,
   output reg ZeroFlag,
   output reg A_GR_B,
   output reg A_LS_B,
   output reg A_EQ_B,
   output reg [7:0] out
);
   reg [8:0] temp;
   reg temps;

   always @ (*)
   begin
      case (sel)
```

```
        2'b00:  // Addition
        begin
          temp = A + B;
                out  = temp[7:0];
        end
        2'b01:  // Subtraction
        begin
          temp = A - B;
          out  = temp[7:0];
        end
        2'b11:  // 2's complement
        begin
          out = ~B + 1;
        end
        default:
        begin
          out = 8'b0;  // Default case for undefined sel value
        end
      endcase

      // Set the flags based on the result
      //OverFlow = A[7] & B[7] & ~out[7];
          OverFlow = (A[7] ^ B[7]) & (out[7] ^ A[7]);
          CarryOut = (A[7] & B[7] & ~out[7]) | (~A[7] & ~B[7] & out[7]);
      ZeroFlag = (out == 0);
      A_GR_B = (A > B) ? 1 : 0;
      A_LS_B = (A < B) ? 1 : 0;
      A_EQ_B = (A == B) ? 1 : 0;
    end

endmodule
```

```
module eightbitshfun(
  input [7:0] A,
  input [1:0] sel,
  output reg [7:0] out
);

always @ (*)
  begin
  case (sel)
    2'b00: out = {A[0], A[7:1]}; // rotate right
    2'b01: out = A[7] ? {A[6:0], 1'b1} : {A[6:0], 1'b0}; // rotate left
    2'b10: out = {1'b0, A[7:1]}; // right shift
    2'b11: out = {A[6:0], 1'b0}; // left shift
    default: out = 8'b0; // default case
  endcase
end

endmodule

module eightbitlu(
  input [7:0] A,
  input [7:0] B,
  input [1:0] sel,
  output reg [7:0] out
);

always @ (*)
  begin
  case (sel)
    2'b00: out = A & B; // and
    2'b01: out = A ^ B; // xor
```

```
   2'b10: out = A | B; // or
   2'b11: out = ~B;    // one's complement of B
   default: out = 8'b0; // default case
  endcase
end

endmodule

module active_low_asynch_rst_dff (
 output reg [0:7] o_q,
 input wire R_W,
 input wire [0:7] i_d,
 input wire i_clk,
 input wire i_rst_n
);

//reg [0:15] data [0:15];
integer i;

always @ (posedge i_clk or negedge i_rst_n)
begin
 if (!i_rst_n)
 begin
 o_q <= 1'b0;
 end
 else
 begin
//  demuxregselect(addr_1,R_W,i_d,data);
//  muxregselect(addr_1,R_W,data,o_q);
```

```
if (R_W)
  begin
   o_q <= i_d;
  end


 end
 end
endmodule

module mux16_8 (
input wire [0:7] A,
input wire [0:7] B,
input wire sel,
output reg [0:7] out
);

always @ (*)
begin

out = (sel) ? A : B;



end
endmodule
```

2.2 ALU 8 Bit Structural module

```
module alu8bit (
input [7:0] X,
input [7:0] Y,
input [3:0] sel,
input i_clk,
input  R_W,
input i_rst_n,
output  [7:0] out,
output  OverFlow,
output  CarryOut,
output  ZeroFlag,
output  A_GR_B,
output  A_LS_B,
output  A_EQ_B
);

wire [7:0] w1;
wire [7:0] w2;
wire [7:0] w3;
wire [7:0] w4;
wire [7:0] w5;

eightBitALU alu(X,Y,{sel[1],sel[0]},OverFlow,CarryOut,ZeroFlag,A_GR_B,A_LS_B,A_EQ_B,w1);
eightbitlu el(X,Y,{sel[1],sel[0]},w2);
eightbitshfun sh(X,{~sel[1],~sel[0]},w3);


mux8b2_1 uy(w2,w3,sel[2],w4);
mux8b2_1 ux(w4,w1,~sel[3],w5);



active_low_asynch_rst_dff rg(out,R_W,w5,i_clk,i_rst_n);



endmodule
```

```
module eightbitshfun(
  input [7:0] A,
  input [1:0] sel,
  output [7:0] out
);

  wire [7:0] w1;
  wire [7:0] w2;
  wire [7:0] shf;
  wire right;
  wire left;

  and8b and1 (.A(A), .B({1'b0, {7{~sel[0]}}}), .out(w1));
  and8b and2 (.A(A), .B({{7{sel[0]}}, 1'b0}), .out(w2));

  buf buf1 (shf[0], w2[1]);
  buf buf2 (shf[7], w1[6]);

  genvar i;
  generate
  for(i=2;i<8;i=i+1) begin : eightbitshfun
  or or2 (shf[i - 1], w1[i - 2], w2[i]);

  end
  endgenerate

  and and3(right,sel[1],A[0],sel[0]);
  and and4(left,sel[1],A[7],~sel[0]);

  or8b or_rot (.A(shf), .B({right,6'b0,left}), .out(out));


endmodule

module eightbitlu(
  input [7:0] A,
  input [7:0] B,
  input [1:0] sel,
  output [7:0] out
);

wire [7:0] w1;
wire [7:0] w2;
wire [7:0] w3;
wire [7:0] w4;

    and8b and1(.A(A),.B(B),.out(w1));
```

```
    xor8b xor1(.A(A),.B(B),.out(w2));
    or8b  or1(.A(A),.B(B),.out(w3));
    not8b not1(.A(B),.out(w4));


    mux8b4x1 mx1(.out(out),.A(w1),.B(w2),.C(w3),.D(w4),.s0(sel[1]),.s1(sel[0]));

endmodule

module eightBitALU (
    input [7:0] A,
    input [7:0] B,
    input [1:0] sel,
    output OverFlow,
    output CarryOut,
    output ZeroFlag,
            output A_GR_B,
            output A_LS_B,
            output A_EQ_B,
            output  [7:0] out
        );

        wire [7:0] out1;
        wire [7:0] out2;
        wire [7:0] out3;
        wire ng,zf,cf,w1,w2,w3;
        wire ng1,zf1,cf1,ww1,ww2,ww3;

        signcom8bit sgn1(.A(A),.B(B),.Cin(sel[0]),.OverFlow(OverFlow),.CarryOut(CarryOut),.ZeroFlag(ZeroFlag),.A_GR_B(A_GR_B),.A_LS_B(A_LS_B),.A_EQ_B(A_EQ_B),.out(out1));

        //signcom8bit sgn2(.A(A),.B(B),.Cin(1'b0),.OverFlow(ng1),.CarryOut(cf1),.ZeroFlag(zf1),.A_GR_B(ww1),.A_LS_B(ww2),.A_EQ_B(ww3),.out(out2));
        signcom8bit sgn3(.A(8'b00000001),.B(~B),.Cin(1'b0),.OverFlow(ng),.CarryOut(cf),.ZeroFlag(zf),.A_GR_B(w1),.A_LS_B(w2),.A_EQ_B(w3),.out(out3));

        mux8b4x1 mx1(.out(out),.A(out1),.B(out1),.C(8'b00000000),.D(out3),.s0(sel[1]),.s1(sel[0]));

        endmodule
```

```
module signcom8bit(
   input  [7:0] A,
   input  [7:0] B,
   input  Cin,
   output OverFlow,
   output CarryOut,
   output ZeroFlag,
   output A_GR_B,
   output A_LS_B,
   output A_EQ_B,
   output [7:0] out
);

wire [7:0] w;
wire [7:0] Bxor;
wire [7:0] zw;
wire zf,wc1,wc2,wc3,wc4;

genvar i;
xor8b xort(.A(B),.B({8{Cin}}),.out(Bxor));

full_adder ad1(.A(A[0]), .B(Bxor[0]), .C(Cin), .sum(out[0]), .cout(w[0]));
generate
for (i = 1; i < 7; i = i + 1 ) begin :fulladder
      full_adder adz(.A(A[i]), .B(Bxor[i]), .C(w[i - 1]), .sum(out[i]), .cout(w[i]));
   end
endgenerate
full_adder ad2(.A(A[7]), .B(Bxor[7]), .C(w[6]), .sum(out[7]), .cout(w[7]));  //for the cout


not8b notz(.A(out),.out(zw));
and8_1b andz(.A(zw),.out(zf));
and and3(ZeroFlag,zf);

and co1(wc1,A[7],B[7],~out[7]);
and co2(wc2,~A[7],~B[7],out[7]);
xor cr(CarryOut,wc1,wc2);

xor xorov1(wc3,A[7],B[7]);
xor xorov2(wc4,out[7],A[7]);
and andov(OverFlow,wc3,wc4);

buf(A_LS_B,(A < B));
buf(A_EQ_B,ZeroFlag);
and and5(A_GR_B,~A_EQ_B,~A_LS_B);
endmodule
```

```
module active_low_asynch_rst_dff (
 output reg [0:7] o_q,
 input wire R_W,
 input wire [0:7] i_d,
 input wire i_clk,
 input wire i_rst_n
);

integer i;

always @ (posedge i_clk or negedge i_rst_n)
begin
 if (!i_rst_n)
 begin
 o_q <= 1'b0;
 end
 else
 begin

if (R_W)
  begin
   o_q <= i_d;
  end


 end
 end
endmodule

module full_adder(
input A,B,C,
output sum,cout
   );
  wire w1,c1,c2,c3,out1;
  xor x1(w1,A,B);
  xor x2(sum,w1,C);

  and a1(c1,A,B);
  and a2(c2,B,C);
  and a3(c3,A,C);

  or o1(out1,c1,c2);
  or o2(cout,out1,c3);
endmodule
```

```
module mux8b4x1 (
  output [7:0] out,
  input  [7:0] A,
  input  [7:0] B,
  input  [7:0] C,
  input  [7:0] D,
  input  s0, s1
);

  genvar i;
  generate
    for (i = 0; i < 8; i = i + 1) begin : mux_4x1
      mux_4x1 mx_inst (
        .out(out[i]),
        .A(A[i]),
        .B(B[i]),
        .C(C[i]),
        .D(D[i]),
        .s0(s0),
        .s1(s1)
      );
    end
  endgenerate

endmodule



module mux8b2_1 (
input [7:0] A,
input [7:0] B,
input sel,
output [7:0] out
);

genvar i;
generate
for(i=0;i<8;i=i+1) begin :  mux2_1
  mux2_1 mx1(.A(A[i]),.B(B[i]),.sel(sel),.out(out[i]));

end
endgenerate
endmodule
```

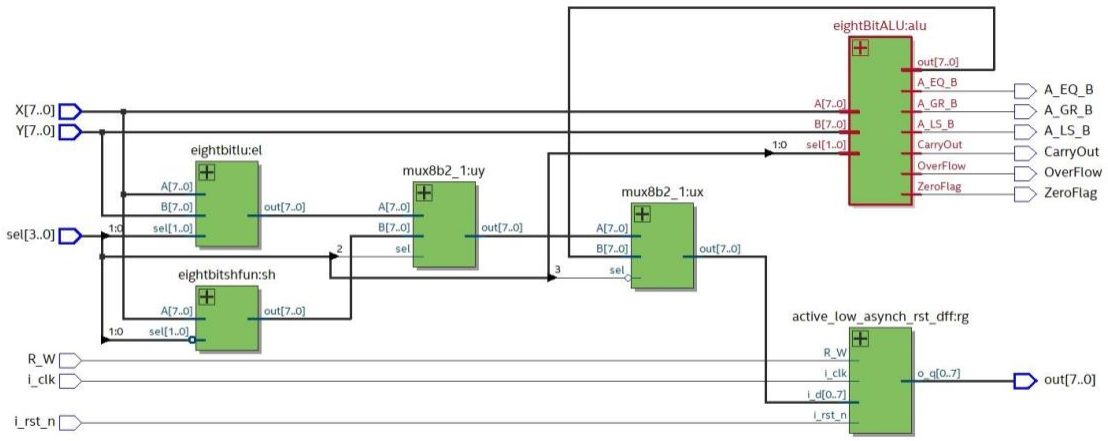
```
module mux2_1 (
input A,
input B,
input sel,
output out
);

wire w1,w2,w3;

not g1(w1,sel);
and g2(w2,w1,A);
and g3(w3,sel,B);
or  g4(out,w2,w3);

endmodule


module mux_4x1(
output out,
input A, B, C, D, s0, s1
);

  wire s0n,s1n; //s0 not, s1 not
  wire w1,w2,w3,w4;

  not n0(s0n,s0);
  not n1(s1n,s1);

  and and1(w1,A,s0n,s1n);
  and and2(w2,B,s0n,s1);
  and and3(w3,C,s0,s1n);
  and and4(w4,D,s0,s1);

  or or1(out,w1,w2,w3,w4);

endmodule


module and8_1b (
input [7:0] A,
output out
);

and and1(out,A[0],A[1],A[2],A[3],A[4],A[5],A[6],A[7]);
endmodule
```

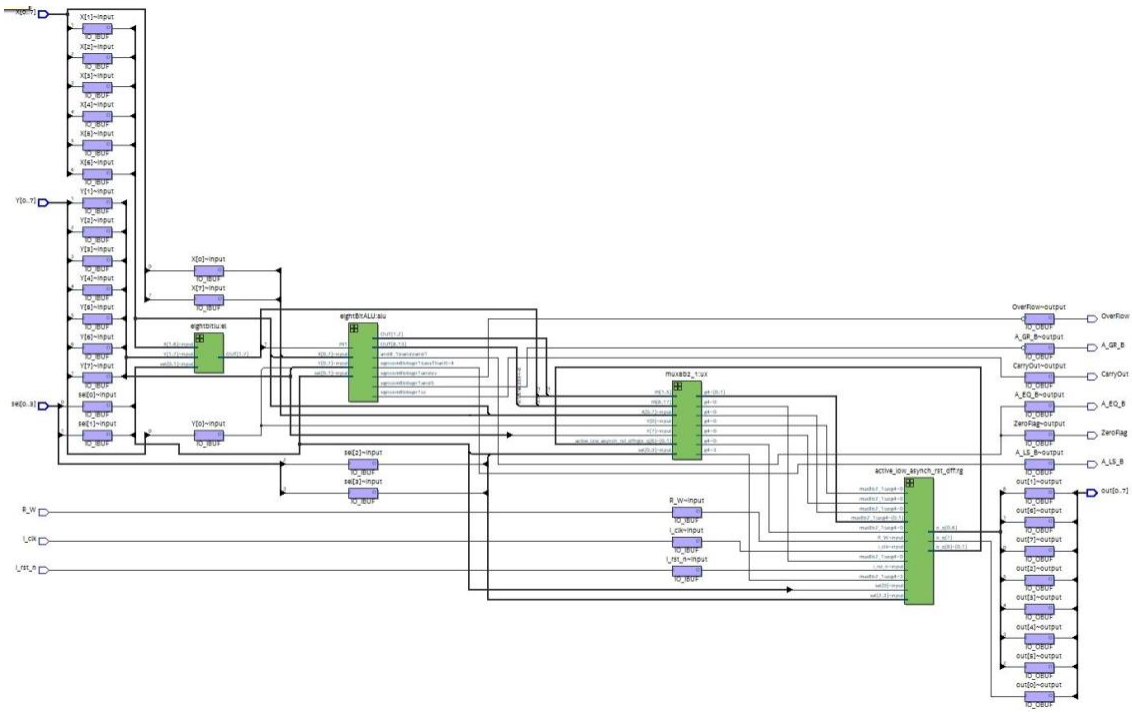
```
module not8b (
input [7:0] A,
output [7:0] out
);


genvar i;
generate
for(i=0;i<8;i=i+1) begin : not8b
not not8(out[i],A[i]);

end
endgenerate
endmodule


module xor8b (
input [7:0] A,
input [7:0] B,
output [7:0] out
);


genvar i;
generate
for(i=0;i<8;i=i+1) begin : xor8b
xor xor8(out[i],A[i],B[i]);

end
endgenerate
endmodule

module or8b (
input [7:0] A,
input [7:0] B,
output [7:0] out
);


genvar i;
generate
for(i=0;i<8;i=i+1) begin : or8b
or or8(out[i],A[i],B[i]);

end
endgenerate
endmodule
```

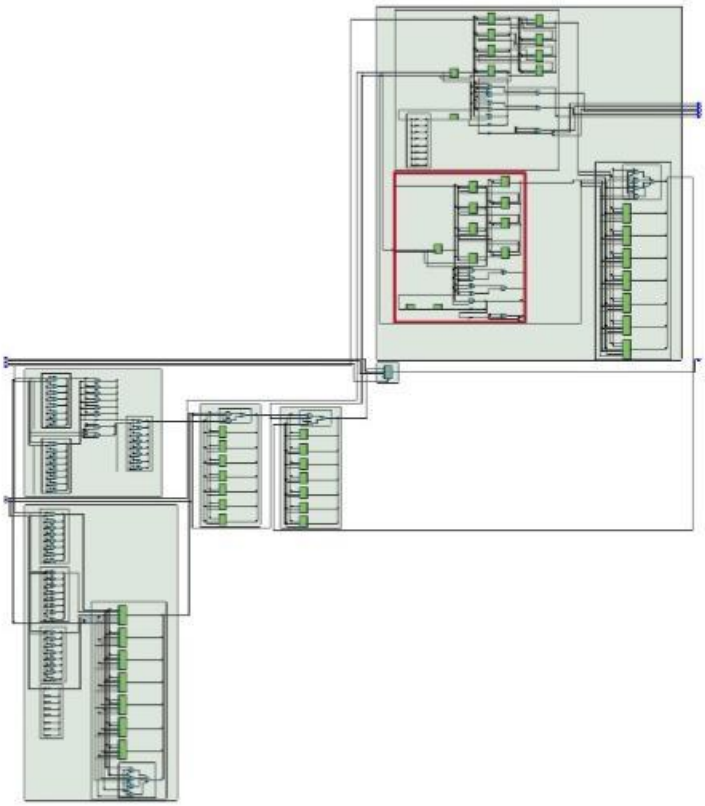
```
module and8b (
input [7:0] A,
input [7:0] B,
output [7:0] out
);


genvar i;
generate
for(i=0;i<8;i=i+1) begin : and8
and and8(out[i],A[i],B[i]);

end
endgenerate
endmodule
```

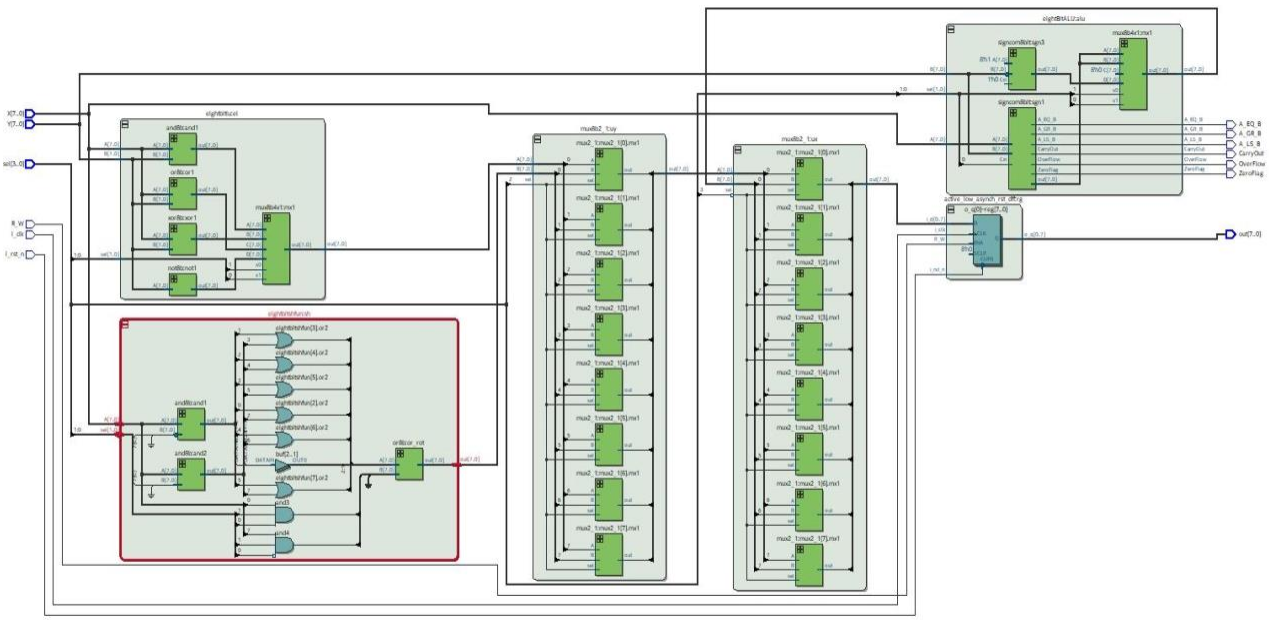
2.3 RTL & Synthesis

RTL

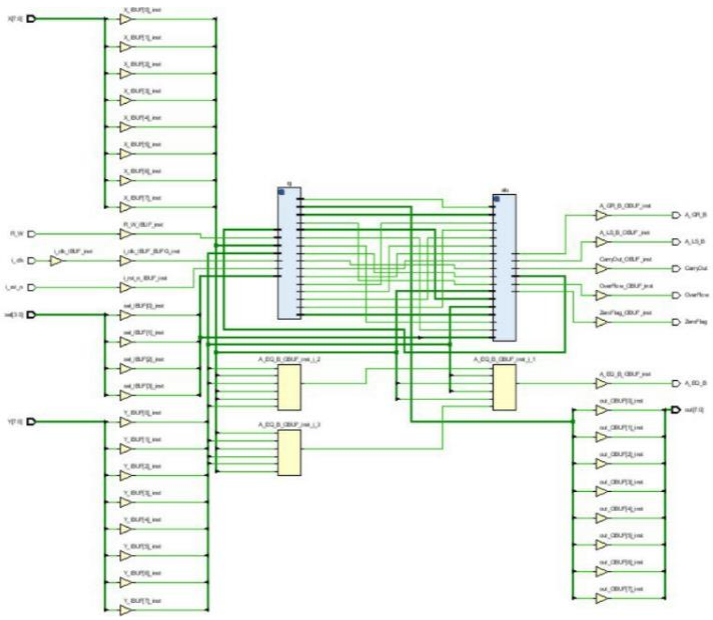








Synthesis



3. Testing & Simulation

3.1 ALU 8_bit testbench

```
module alu8bitest();

    reg [7:0] X;
    reg [7:0] Y;
    reg [3:0] sel;
    reg i_clk;
    reg R_W;
    reg i_rst_n;
    wire OverFlow;
    wire CarryOut;
    wire ZeroFlag;
    wire A_GR_B;
    wire A_LS_B;
    wire A_EQ_B;
    wire [7:0] out;

alu8bit test(X,Y,sel,i_clk,R_W,i_rst_n,out,OverFlow,CarryOut,ZeroFlag,A_GR_B,A_LS_B,A_EQ_B);

initial begin
  i_clk=0;

    forever #1 i_clk = ~i_clk;
end

initial begin
  sel=4'b0;

    forever #4 sel = sel + 1;
end


initial
begin

X=8'b0;
Y=8'b0;
sel=4'b0000;
R_W=0;
i_rst_n=0;
```

```
40.
41.
42.         $dumpfile("voter_tb.vcd");  // Waveform file
43.         $dumpvars(0, test);         // signals to plot (everything)
44.
45.         $display("X Y sel  | out OF CF ZF A<B A>B A=B ");
46.         $display("-------------------------");
47.
48.         $monitor("%b %b %b  | %b %b %b %b %b %b
    %b",X,Y,sel,out,OverFlow,CarryOut,ZeroFlag,A_LS_B,A_GR_B,A_EQ_B);
49.
50.
51. #1
52. i_rst_n=1;
53. R_W=1;
54.
55.
56. X=8'b00001111;
57. Y=8'b11110000;
58. #3
59. check(8'b11111111,3'b100);
60.
61. X=8'b00001111;
62. Y=8'b11110000;
63. #3
64. check(8'b00011111,3'b000);
65.
66.
67. X=8'b00101000;
68. Y=8'b00010000;
69. #7
70. check(8'b11110000,3'bxxx);
71.
72.                 //And
73. X=8'b01100101;
74. Y=8'b00100101;
75. #22
76. check(8'b00100101,3'bxxx);
77.
78.                 //Xor
79. X=8'b01100111;
80. Y=8'b00100100;
81. #3
82. check(8'b01000011,3'bxxx);
```

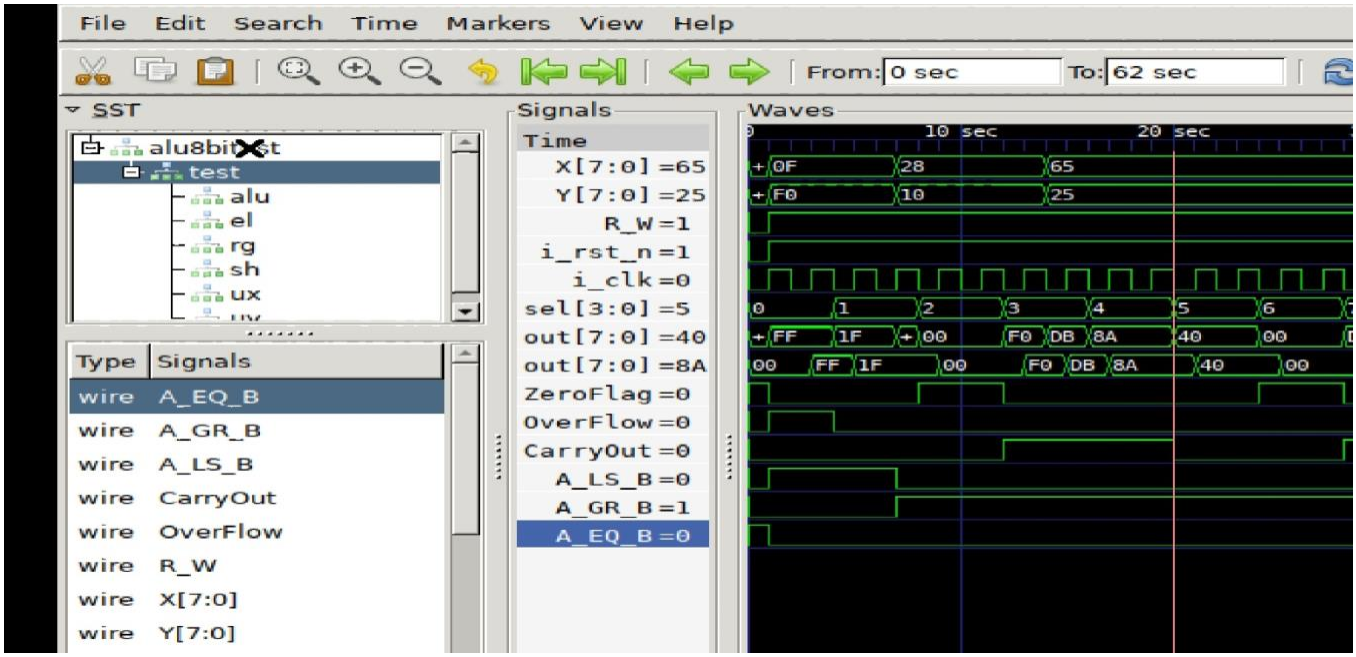
```
83.         //Or
84. X=8'b01100101;
85. Y=8'b00100100;
86. #3
87. check(8'b01100101,3'bxxx);
88.
89.                     //Not (1's complement)
90. X=8'b01010101;
91. Y=8'b01010101;
92. #4
93. check(8'b10101010,3'bxxx);
94.
95.                     //Rotate right
96. X=8'b01100101;
97. Y=8'b00100100;
98. #4
99. check(8'b10110010,3'bxxx);
100.
101.                            //Rotate left
102.            X=8'b01100101;
103.            Y=8'b00100100;
104.            #4
105.            check(8'b11001010,3'bxxx);
106.
107.                            //Right shift
108.            X=8'b01100101;
109.            Y=8'b00100100;
110.            #4
111.            check(8'b00110010,3'bxxx);
112.
113.                            //Left shift
114.            X=8'b01100101;
115.            Y=8'b00100100;
116.            #4
117.            check(8'b11001010,3'bxxx);
118.
119.
120.              $finish;
121.            end
122.
123.
124.            task check(input [7:0] out_exp, input [2:0] flags);
125.            begin
126.
127.            if(sel < 4'b0011) begin
128.
129.            if({out_exp,flags} == {out,OverFlow,CarryOut,ZeroFlag})
```

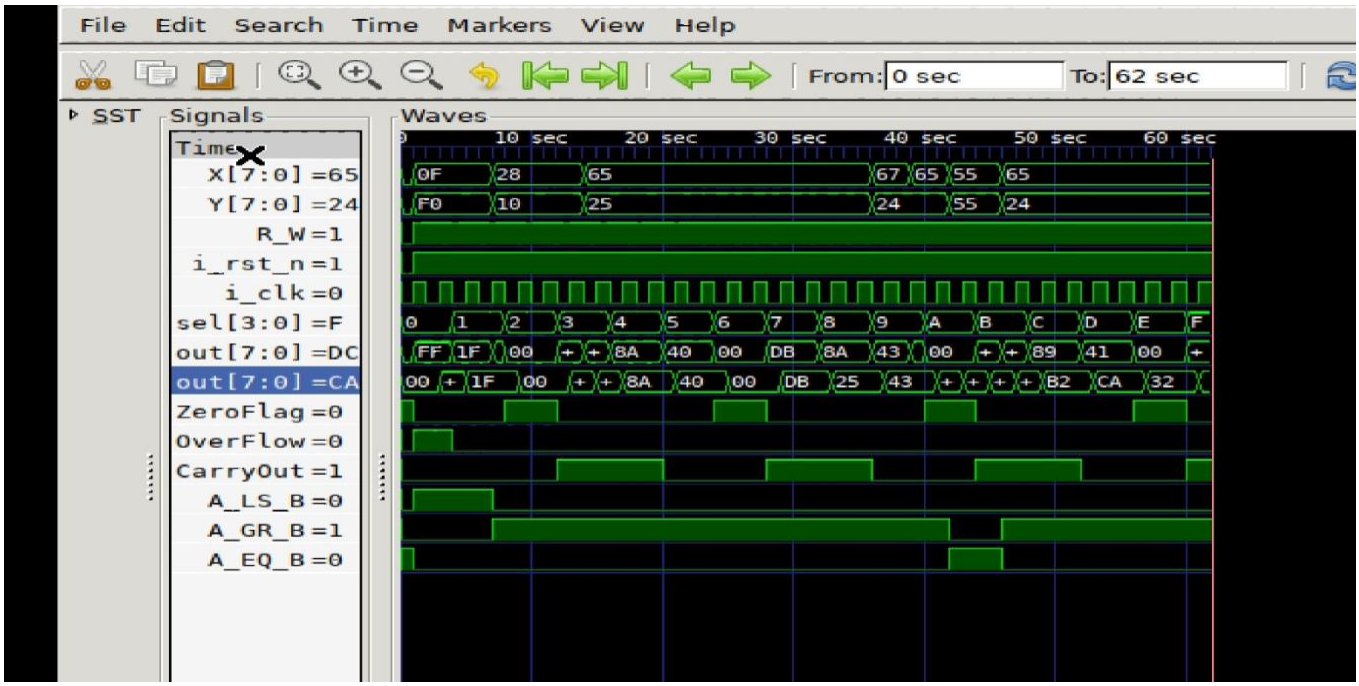
```
$display("test successfull out is %b out_exp is %b flags %b %b",out,out_exp,flags,{OverFlow,CarryOut,ZeroFlag});

else
$display("test failed out %b out_exp %b flags %b %btime %0t",out,out_exp,flags,{OverFlow,CarryOut,ZeroFlag},$time);
end

else begin

if(out_exp == out)
$display("test successfull out is %b out_exp is %b",out,out_exp);

else
$display("test failed out %b out_exp %b time %0t",out,out_exp,$time);
end

end
endtask

endmodule
```

Testbench output

```
X Y sel  | out OF CF ZF A<B A>B A=B
-------------------------
00000000 00000000 0000  | 00000000 0 0 1 0 0 1
00001111 11110000 0000  | 00000000 1 0 0 1 0 0
00001111 11110000 0000  | 11111111 1 0 0 1 0 0
test successfull out is 11111111 out_exp is 11111111 flags 100 100
00001111 11110000 0001  | 11111111 0 0 0 1 0 0
00001111 11110000 0001  | 00011111 0 0 0 1 0 0
test successfull out is 00011111 out_exp is 00011111 flags 000 000
00101000 00010000 0001  | 00011111 0 0 0 0 1 0
00101000 00010000 0010  | 00011111 0 0 1 0 1 0
00101000 00010000 0010  | 00000000 0 0 1 0 1 0
00101000 00010000 0011  | 00000000 0 1 0 0 1 0
00101000 00010000 0011  | 11110000 0 1 0 0 1 0
test successfull out is 11110000 out_exp is 11110000
01100101 00100101 0011  | 11110000 0 1 0 0 1 0
01100101 00100101 0011  | 11011011 0 1 0 0 1 0
01100101 00100101 0100  | 11011011 0 1 0 0 1 0
01100101 00100101 0100  | 10001010 0 1 0 0 1 0
01100101 00100101 0101  | 10001010 0 0 0 0 1 0
01100101 00100101 0101  | 01000000 0 0 0 0 1 0
01100101 00100101 0110  | 01000000 0 0 1 0 1 0
01100101 00100101 0110  | 00000000 0 0 1 0 1 0
01100101 00100101 0111  | 00000000 0 1 0 0 1 0
01100101 00100101 0111  | 11011011 0 1 0 0 1 0
01100101 00100101 1000  | 11011011 0 1 0 0 1 0
01100101 00100101 1000  | 00100101 0 1 0 0 1 0
test successfull out is 00100101 out_exp is 00100101
01100111 00100100 1001  | 00100101 0 0 0 0 1 0
01100111 00100100 1001  | 01000011 0 0 0 0 1 0
test successfull out is 01000011 out_exp is 01000011
01100101 00100100 1001  | 01000011 0 0 0 0 1 0
01100101 00100100 1010  | 01000011 0 0 1 0 1 0
01100101 00100100 1010  | 01100101 0 0 1 0 1 0
test successfull out is 01100101 out_exp is 01100101
01010101 01010101 1010  | 01100101 0 0 1 0 0 1
01010101 01010101 1010  | 01010101 0 0 1 0 0 1
01010101 01010101 1011  | 01010101 0 1 0 0 0 1
01010101 01010101 1011  | 10101010 0 1 0 0 0 1
test successfull out is 10101010 out_exp is 10101010
01100101 00100100 1011  | 10101010 0 1 0 0 1 0
01100101 00100100 1011  | 11011011 0 1 0 0 1 0
01100101 00100100 1100  | 11011011 0 1 0 0 1 0
01100101 00100100 1100  | 10110010 0 1 0 0 1 0
test successfull out is 10110010 out_exp is 10110010
01100101 00100100 1101  | 10110010 0 0 0 0 1 0
01100101 00100100 1101  | 11001010 0 0 0 0 1 0
test successfull out is 11001010 out_exp is 11001010
01100101 00100100 1110  | 11001010 0 0 1 0 1 0
01100101 00100100 1110  | 00110010 0 0 1 0 1 0
test successfull out is 00110010 out_exp is 00110010
01100101 00100100 1111  | 00110010 0 1 0 0 1 0
01100101 00100100 1111  | 11001010 0 1 0 0 1 0
test successfull out is 11001010 out_exp is 11001010
Alutest.v:122: $finish called at 62 (1s)
```

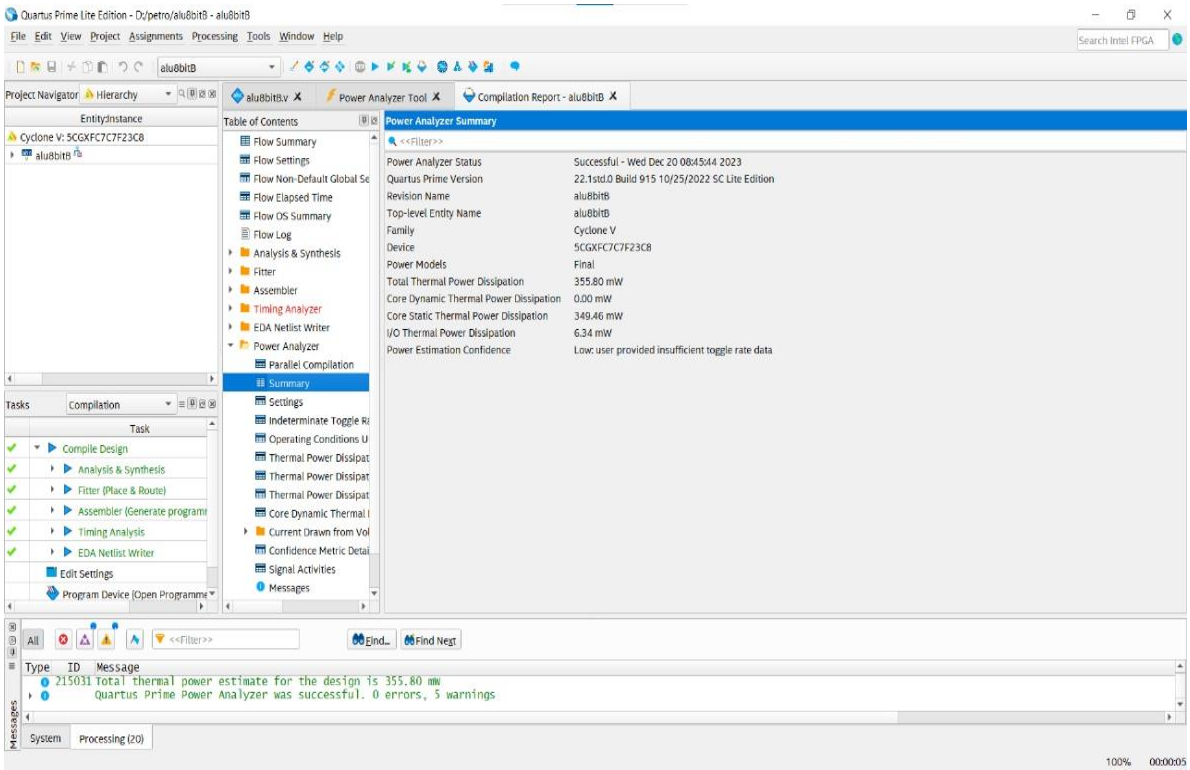
3.2 Simulation Waveform output

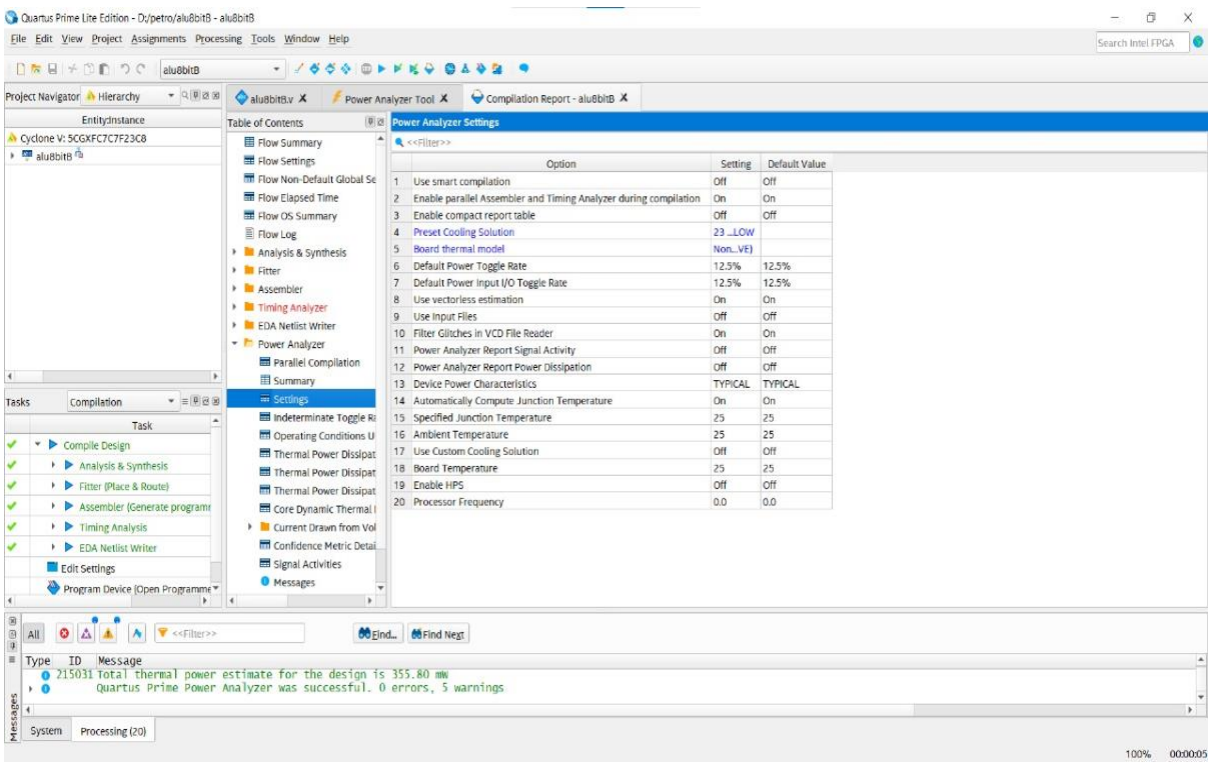


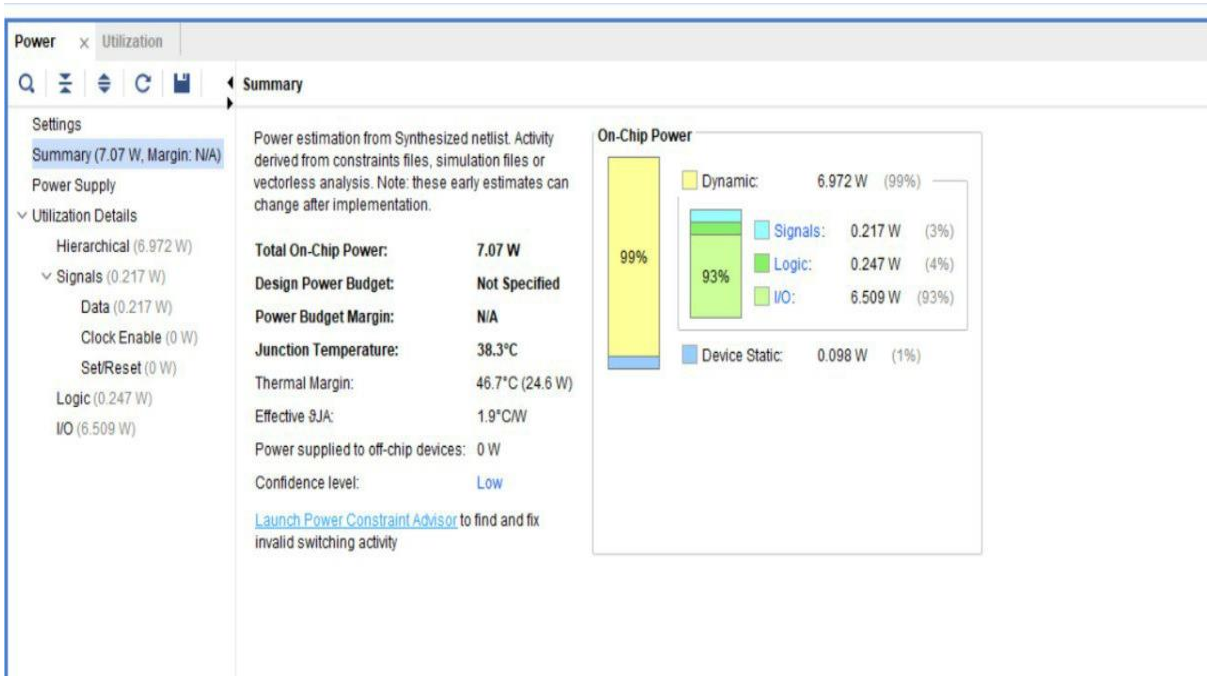


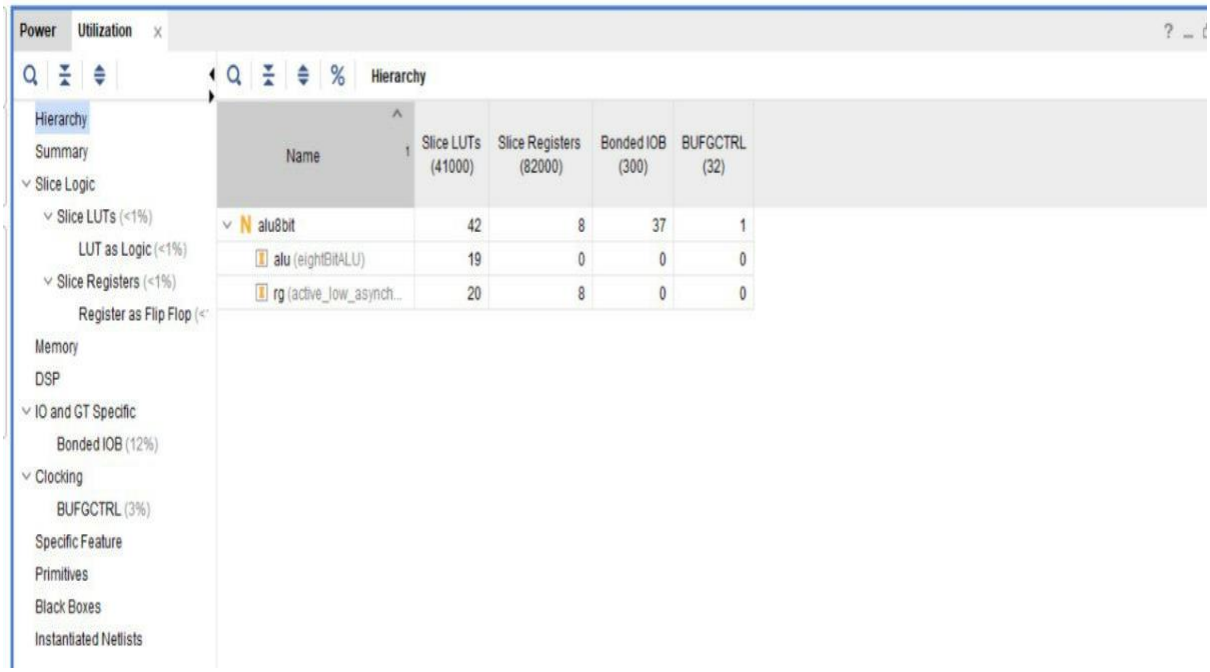
4. Power & Timing analysis

Power



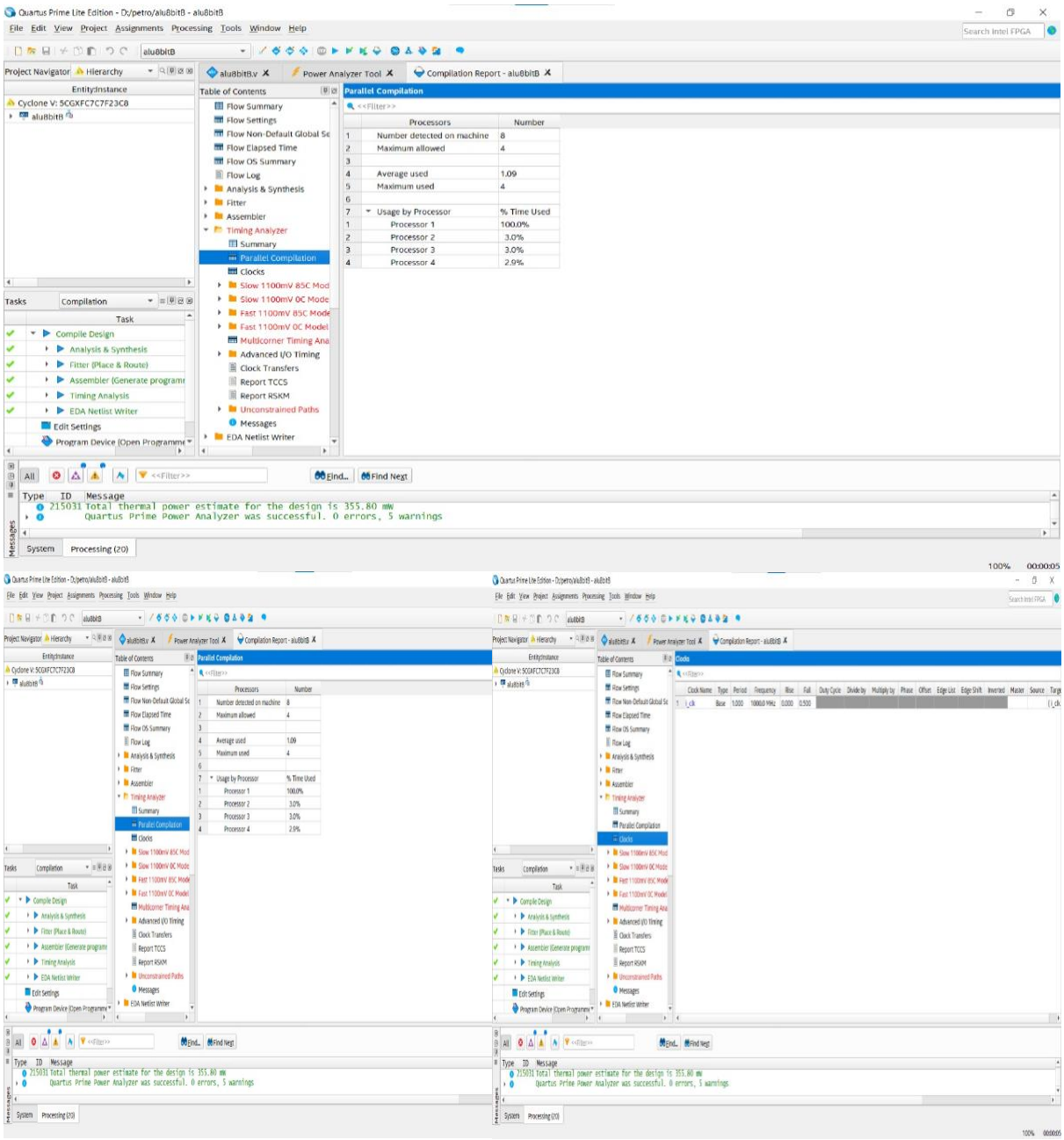






Name	Slice LUTs (41000)	Slice Registers (82000)	Bonded IOB (300)	BUFGCTRL (32)
alu8bit	42	8	37	1
alu (eightBitALU)	19	0	0	0
rg (active_low_asynch...	20	8	0	0

Timing


Quartus Prime Lite Edition - D:/petro/alu8bitB - alu8bitB
File Edit View Project Assignments Processing Tools Window Help
Project Navigator Hierarchy
Entity:Instance
Cyclone V: 5CGXFC7C7F23C8
alu8bitB
alu8bitB.v
Power Analyzer Tool
Compilation Report - alu8bitB
Table of Contents
Flow Summary
Flow Settings
Flow Non-Default Global Se
Flow Elapsed Time
Flow OS Summary
Flow Log
Analysis & Synthesis
Fitter
Assembler
Timing Analyzer
Summary
Parallel Compilation
Clocks
Slow 1100mV 85C Mod
Slow 1100mV 0C Mode
Fast 1100mV 85C Mode
Fast 1100mV 0C Model
Multicorner Timing Ana
Advanced I/O Timing
Clock Transfers
Report TCCS
Report RSKM
Unconstrained Paths
Messages
EDA Netlist Writer
Parallel Compilation
Processors Number
1 Number detected on machine 8
2 Maximum allowed 4
3
4 Average used 1.09
5 Maximum used 4
6
7 Usage by Processor % Time Used
1 Processor 1 100.0%
2 Processor 2 3.0%
3 Processor 3 3.0%
4 Processor 4 2.9%
Tasks Compilation
Task
Compile Design
Analysis & Synthesis
Fitter (Place & Route)
Assembler (Generate programi
Timing Analysis
EDA Netlist Writer
Edit Settings
Program Device (Open Programme
Find... Find Next
Type ID Message
215031 Total thermal power estimate for the design is 355.80 mW
Quartus Prime Power Analyzer was successful. 0 errors, 5 warnings
System Processing (20)

5. References

[1] Arithmetic logic Units/Function Generators Datasheet ,Texas Instrument, Dallas ,Texas 19888 [Online] Available http://www-s.ti.com/sc/ds/sn54s181.pdf